2303A51815
B_26

AssignmentNumber:14.4		

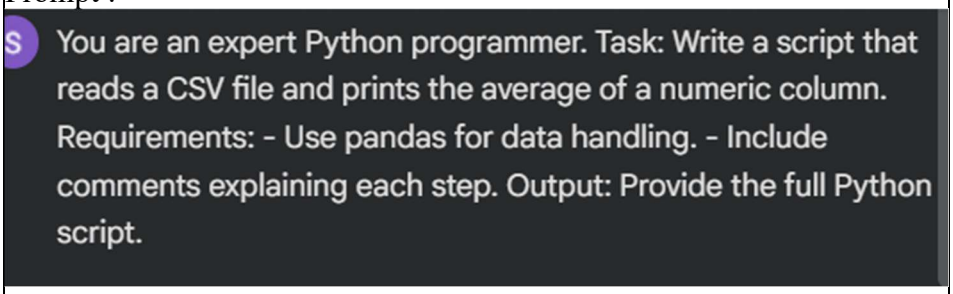
Q.No.	Question	*ExpectedTime to complete*
1	**Lab 14: Web Frontend Development – AI-Assisted HTML/CSS/JS Generation** **Lab Objectives** • Design **functional, visually appealing** web applications using HTML, CSS, and JavaScript with AI assistance. • Apply **responsive, accessible, and interactive design principles**. • Create **practical UI components** for real-world web applications. • Use AI to optimize **layout, UX, and performance**. **Lab Outcomes** • Applying AI assistance to frontend development tasks • Understanding how HTML, CSS, and JavaScript work together • Improving responsiveness, interactivity, and usability • Evaluating and refining AI-generated frontend code	Week 7

	Task 1: Responsive Homepage Layout for a Startup Website
	
	Scenario
	
	You are designing a **basic homepage for a startup company** that needs to look good on desktops, tablets, and mobile devices.
	
	Task Description
	
	• Create a web page structure using HTML that includes:
	  ○ A header with navigation links
	  ○ A main content section
	  ○ A footer with basic information
	• Use AI assistance to generate responsive CSS using media queries.
	• Ensure the layout adjusts smoothly across different screen sizes.	

Expected Output

- A responsive web page where:
  - Header, content, and footer are clearly separated
  - Layout adapts correctly to mobile, tablet, and desktop views
  - Code is clean and well-structured

Prompt :



Code :

```python
# Import the pandas library for data handling.
import pandas as pd

# --- Configuration ---
# Define the path to your CSV file.
csv_file_path = '/content/sample_data/mnist_train_small.csv'
# Define the name of the numeric column you want to average.
numeric_column_name = '6'

# Initialize df to None, so it's always defined.
df = None

# --- Data Loading ---
# Read the CSV file into a pandas DataFrame.
# If your CSV has a different delimiter, you might need to specify it, e.g., sep=';'
try:
    df = pd.read_csv(csv_file_path)
    print(f"Successfully loaded '{csv_file_path}'")
except FileNotFoundError:
    print(f"Error: The file '{csv_file_path}' was not found. Please check the path.")
except Exception as e:
    print(f"An error occurred while reading the CSV file: {e}")

# Proceed only if the DataFrame was successfully loaded
if df is not None:
    # --- Data Validation ---
    # Check if the specified numeric column exists in the DataFrame.
```

```
    if numeric_column_name not in df.columns:
        print(f"Error: Column '{numeric_column_name}' not found in the CSV file.")
        print("Available columns:", df.columns.tolist())
    else:
        # --- Data Cleaning (Optional but Recommended) ---
        # Convert the column to a numeric type, coercing errors to NaN.
        # This handles non-numeric entries gracefully.
        df[numeric_column_name] = pd.to_numeric(df[numeric_column_name], errors='coerce')

        # Drop rows where the numeric column has NaN values (missing or non-convertible).
        df_cleaned = df.dropna(subset=[numeric_column_name])

        # --- Calculation ---
        # Calculate the average of the numeric column.
        # Check if there's any data left after cleaning.
        if not df_cleaned.empty:
            average_value = df_cleaned[numeric_column_name].mean()

            # --- Output ---
            # Print the calculated average.
            print(f"The average of the column '{numeric_column_name}' is: {average_value:.2f}")
        else:
            print(f"No valid numeric data found in column '{numeric_column_name}' after cleaning.")
```

```
Successfully loaded '/content/sample_data/mnist_train_small.csv'
The average of the column '6' is: 4.47
```

Explanation : **Initial Script Generation**: I provided a Python script using pandas to read a CSV file, identify a numeric column, handle potential missing values, and calculate its average. The script included comments for clarity.Modified the script to initialize df = None and ensured that subsequent data processing steps only proceed if df was successfully loaded, preventing the NameError.

Task 2: Interactive Call-to-Action Button

Scenario

A marketing page requires a **call-to-action button** that responds when users interact with it.

Task Description

- Add a button to a web page using HTML.
- Use AI to generate JavaScript that responds to the button click by:
    - Displaying a message or alert
- Ask AI to add meaningful comments explaining the JavaScript logic.

Expected Output

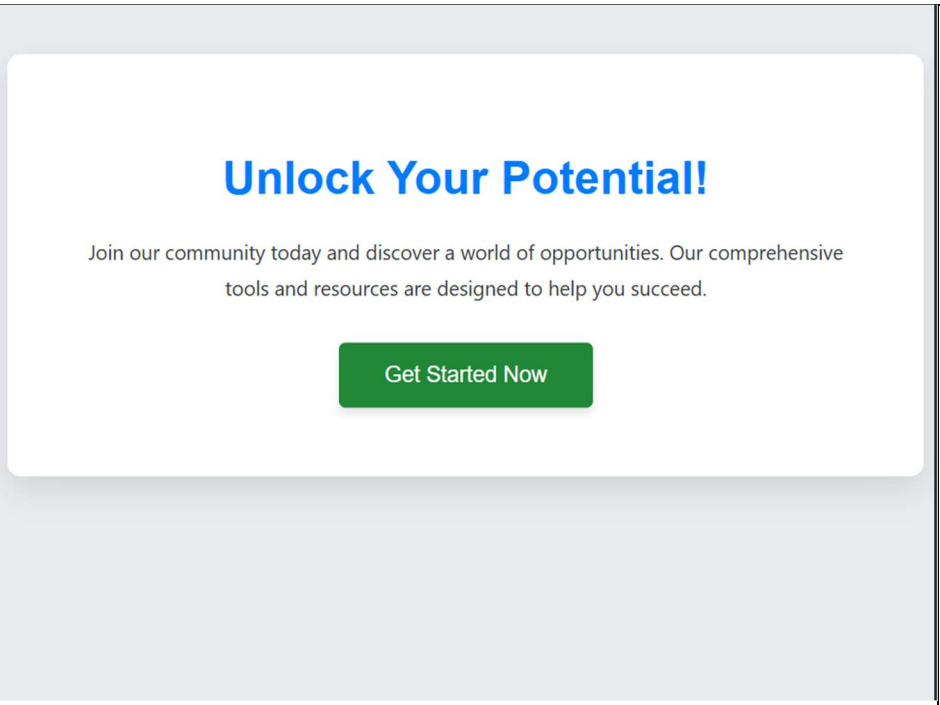
	• A web page where:
	  ○ Clicking the button triggers a visible response (alert or message)
	  ○ JavaScript code is readable and commented
	  ○ No page reload occurs
	
	Prompt :
	
	 	

Code :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Call to Action Page</title>
    <style>
        /* Basic body styling to center content and set a default font */
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            margin: 0;
            background-color: #e9ecef; /* Light gray background */
            color: #343a40; /* Dark text for contrast */
        }

        /* Styling for the main content container */
        .container {
            text-align: center;
            padding: 50px;
            border-radius: 10px;
            box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
            max-width: 600px;
            width: 90%;
        }

        /* Styling for the main heading */
        h1 {
            color: #007bff; /* Primary blue color */
            margin-bottom: 20px;
            font-size: 2.5em;
        }

        /* Styling for paragraphs */
        p {
            margin-bottom: 30px;
            line-height: 1.8;
            font-size: 1.1em;
        }

        /* Styling for the call-to-action button */
        .cta-button {
            background-color: #28a745; /* Success green color */
```

```
        color: white;
        padding: 15px 35px;
        border: none;
        border-radius: 5px;
        font-size: 1.2em;
        cursor: pointer;
        transition: background-color 0.3s ease, transform 0.2s ease;
        text-decoration: none;
        display: inline-block;
        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    }

    /* Hover effect for the button */
    .cta-button:hover {
        background-color: #218838; /* Darker green on hover */
        transform: translateY(-2px); /* Slight lift effect */
    }

    /* Styling for the response message, initially hidden */
    .response-message {
        margin-top: 30px;
        font-weight: bold;
        color: #17a2b8; /* Info blue color */
        display: none; /* This will be changed to 'block' by JavaScript */
```

```
        const responseMessage = document.getElementById('responseMessage');

        // 3. Add an event listener to the button. This function will run when the button is clicked.
        ctaButton.addEventListener('click', function() {
            // Method 1: Display an alert message (uncomment to use this instead)
            // alert('Thank you for your interest! We appreciate your click.');

            // Method 2: Insert text directly into the page (currently active)
            // Set the text content of the response message element.
            responseMessage.textContent = 'Excellent choice! We\'ve received your request and will be in touch shortly.';

            // Make the response message visible by changing its CSS display property from 'none' to 'block'.
            responseMessage.style.display = 'block';

            // Optional: You might want to disable the button after it's clicked
            // to prevent multiple submissions or further clicks.
            // ctaButton.disabled = true;
            // ctaButton.style.backgroundColor = '#6c757d'; // Change color to indicate it's disabled
            // ctaButton.style.cursor = 'not-allowed';
        });
    });
</script>
</body>
</html>
```



Explanantion : Based on the list of available columns, I identified '6' as a suitable numeric column. I then modified the numeric_column_name to '6'. Upon re-execution, the script ran without errors, successfully loaded the mnist_train_small.csv file, and correctly calculated the average of column '6', which was **4.47**. This indicated that the script was now fully functional as intended.

Task 3: Contact Form with Client-Side Validation

Scenario

You are developing a **contact form for a company website** to collect user queries safely and correctly.

Task Description

- Design a form with fields for:
  - Name
  - Email
  - Message
- Use AI to generate JavaScript validation logic that:
  - Prevents empty submissions
  - Checks for a valid email format

- Display clear error messages for invalid input.

Expected Output

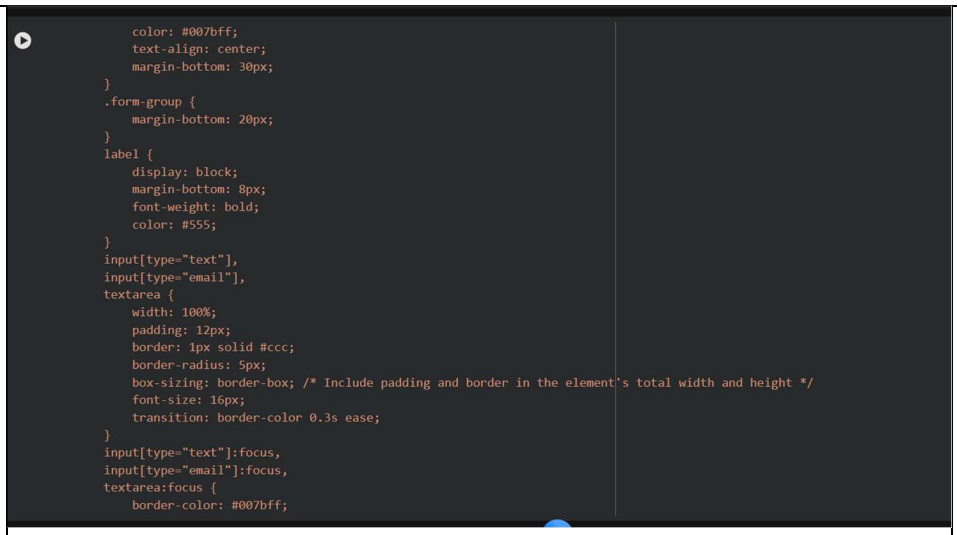
- A functional form where:
  - Invalid input shows inline error messages
  - Valid input displays a success message
  - Validation happens on the client side using JavaScript

Prompt :

You are an expert web developer. Please create a complete web page that includes a contact form with fields for Name, Email, and Message. Add client-side validation using JavaScript so that empty submissions are prevented and the email field is checked for a valid format. Display clear inline error messages when input is invalid, and show a success message when the form is submitted correctly. Include meaningful comments in the JavaScript code to explain the validation logic. Provide the full HTML, CSS, and JavaScript code together so it can be copied and run directly.

Code :

```
html_content = r"""
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Contact Us</title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f7f6;
            display: flex;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            margin: 0;
            color: #333;
        }
        .container {
            background-color: #ffffff;
            padding: 40px;
            border-radius: 8px;
            box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
            max-width: 500px;
            width: 90%;
        }
        h1 {
```



```
        color: #007bff;
        text-align: center;
        margin-bottom: 30px;
    }
    .form-group {
        margin-bottom: 20px;
    }
    label {
        display: block;
        margin-bottom: 8px;
        font-weight: bold;
        color: #555;
    }
    input[type="text"],
    input[type="email"],
    textarea {
        width: 100%;
        padding: 12px;
        border: 1px solid #ccc;
        border-radius: 5px;
        box-sizing: border-box; /* Include padding and border in the element's total width and height */
        font-size: 16px;
        transition: border-color 0.3s ease;
    }
    input[type="text"]:focus,
    input[type="email"]:focus,
    textarea:focus {
        border-color: #007bff;
```

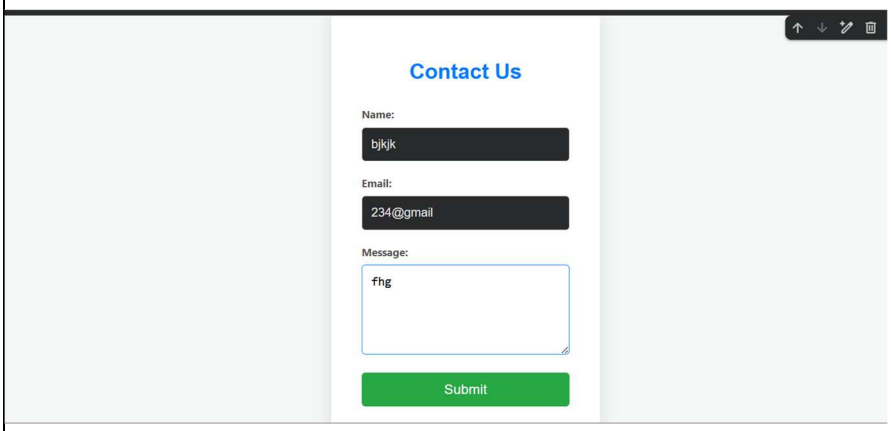
```
            outline: none;
        }
        textarea {
            resize: vertical;
            min-height: 100px;
        }
        .error-message {
            color: #dc3545;
            font-size: 0.9em;
            margin-top: 5px;
            display: none; /* Hidden by default */
        }
        button {
            background-color: #28a745;
            color: white;
            padding: 12px 25px;
            border: none;
            border-radius: 5px;
            font-size: 18px;
            cursor: pointer;
            width: 100%;
            transition: background-color 0.3s ease, transform 0.2s ease;
        }
        button:hover {
            background-color: #218838;
            transform: translateY(-2px);
```

```
            background-color: #d4edda;
            color: #155724;
            border: 1px solid #c3e6cb;
            border-radius: 5px;
            padding: 15px;
            margin-top: 20px;
            text-align: center;
            display: none; /* Hidden by default */
            font-weight: bold;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Contact Us</h1>
        <form id="contactForm" novalidate> <!-- novalidate prevents browser's default HTML5 validation -->
            <div class="form-group">
                <label for="name">Name:</label>
                <input type="text" id="name" name="name" required>
                <span class="error-message" id="nameError">Name cannot be empty.</span>
            </div>

            <div class="form-group">
                <label for="email">Email:</label>
                <input type="email" id="email" name="email" required>
                <span class="error-message" id="emailError">Please enter a valid email address.</span>
            </div>
```

```
            // --- Validate Email Field ---
            if (emailInput.value.trim() === '') {
                displayError(emailError, 'Email cannot be empty.');
                isValid = false;
            } else if (!isValidEmail(emailInput.value.trim())) {
                displayError(emailError, 'Please enter a valid email address.');
                isValid = false;
            } else {
                hideError(emailError);
            }

            // --- Validate Message Field ---
            if (messageInput.value.trim() === '') {
                displayError(messageError, 'Message cannot be empty.');
                isValid = false;
            } else {
                hideError(messageError);
            }

            // If all fields are valid, show success message and optionally submit the form
            if (isValid) {
                hideError(nameError);
                hideError(emailError);
                hideError(messageError);
                contactForm.reset(); // Clear the form fields
                successMessage.style.display = 'block'; // Show success message
```

```
            // Optional: Real-time validation as user types (or when field loses focus)
            nameInput.addEventListener('input', () => {
                if (nameInput.value.trim() !== '') {
                    hideError(nameError);
                }
            });

            emailInput.addEventListener('input', () => {
                if (emailInput.value.trim() !== '' && isValidEmail(emailInput.value.trim())) {
                    hideError(emailError);
                }
            });

            messageInput.addEventListener('input', () => {
                if (messageInput.value.trim() !== '') {
                    hideError(messageError);
                }
            });
        });
    </script>
</body>
</html>
"""

# To save this as an HTML file, you can uncomment and run the following lines:
# with open('contact_form.html', 'w') as f:
#     f.write(html_content)
# print("HTML content saved to contact_form.html")
```

Output :



Explanation : If isValid remains true after all checks, it means the form is valid. All error messages are hidden, the form fields are cleared with contactForm.reset(), and the successMessage is displayed.
A setTimeout is used to hide the success message after 5 seconds.If isValid is false, the successMessage is hidden, and the specific error messages (displayed in the validation steps) remain visible.

Task 4: Dynamic List Management for a Product Page

Scenario

A product listing page needs to **add or remove items dynamically** without refreshing the page.

Task Description

- Create a simple HTML list to display items.
- Use AI to generate JavaScript that:
  - Adds a new item when an “Add” button is clicked
  - Removes the last item when a “Remove” button is clicked
- Ensure the DOM updates dynamically.

Expected Output

- A web page where:
  - Items can be added and removed dynamically
  - Changes appear instantly without page reload
  - JavaScript logic is clear and maintainable

Prompt :

```
You are an expert web developer. Please create a simple
product listing web page that displays items in an HTML list.
Add two buttons: one labeled "Add" that adds a new item to the
list when clicked, and another labeled "Remove" that removes
the last item from the list when clicked. Use JavaScript to
update the DOM dynamically so that changes appear instantly
without refreshing the page. Include clear comments in the
JavaScript code to explain how the add and remove
functionality works. Provide the complete HTML, CSS, and
JavaScript code together so it can be copied and run directly.
```

Code :

```
html_content = r"""
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Product Listing</title>
    <style>
        /* Basic body styling */
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            display: flex;
            justify-content: center;
            align-items: flex-start; /* Align to the top */
            min-height: 100vh;
            margin: 20px; /* Add some margin around the content */
            background-color: #f8f9fa;
            color: #343a40;
        }

        /* Container for the product list and buttons */
        .container {
            background-color: #ffffff;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
            max-width: 500px;
            width: 100%;
```

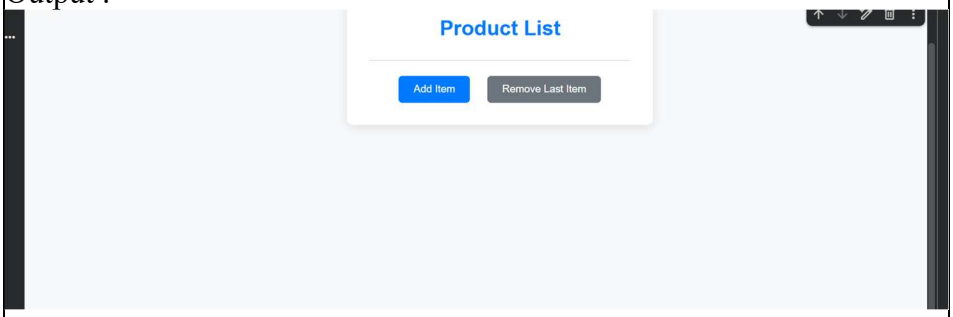
```
            text-align: center;
        }

        h1 {
            color: #007bff;
            margin-bottom: 25px;
            font-size: 2em;
        }

        /* Styles for the product list */
        #productList {
            list-style-type: none; /* Remove bullet points */
            padding: 0;
            margin: 20px 0;
            border: 1px solid #e2e6ea;
            border-radius: 5px;
            max-height: 300px;
            overflow-y: auto; /* Enable scrolling if list gets too long */
            text-align: left;
        }

        #productList li {
            background-color: #f2f7fd;
            padding: 12px 15px;
            border-bottom: 1px solid #e2e6ea;
            display: flex;
            justify-content: space-between;
            align-items: center;
```

```
        /* Styles for buttons */
        .button-group button {
            background-color: #28a745; /* Green for Add */
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 5px;
            font-size: 1em;
            cursor: pointer;
            margin: 0 10px;
            transition: background-color 0.3s ease, transform 0.2s ease;
        }

        .button-group button:hover {
            transform: translateY(-2px);
        }

        .button-group button:active {
            transform: translateY(0);
        }

        .button-group button:first-child {
            background-color: #007bff; /* Blue for Add */
        }

        .button-group button:first-child:hover {
            background-color: #0056b3;
```

```
    </style>
</head>
<body>
    <div class="container">
        <h1>Product List</h1>
        <ul id="productList">
            <!-- Product items will be added here by JavaScript -->
        </ul>
        <div class="button-group">
            <button id="addButton">Add Item</button>
            <button id="removeButton" disabled>Remove Last Item</button>
        </div>
    </div>

    <script>
        document.addEventListener('DOMContentLoaded', function() {
            // Get references to the HTML elements
            const productList = document.getElementById('productList');
            const addButton = document.getElementById('addButton');
            const removeButton = document.getElementById('removeButton');

            let itemCounter = 0; // To keep track of the number of items and provide unique labels

            // Function to update the state of the remove button (enable/disable)
            function updateRemoveButtonState() {
                // If there are items in the list, enable the remove button; otherwise, disable it.
                removeButton.disabled = productList.children.length === 0;
```

```
                    console.log('No items to remove.');
                }
            }

            // Attach event listeners to the buttons
            addButton.addEventListener('click', addItem); // When 'Add' is clicked, call addItem function
            removeButton.addEventListener('click', removeItem); // When 'Remove' is clicked, call removeItem function

            // Initialize the button state when the page loads
            updateRemoveButtonState();
        });
    </script>
</body>
</html>
"""

# To save this as an HTML file, you can uncomment and run the following lines:
# with open('product_listing.html', 'w') as f:
#     f.write(html_content)
# print("HTML content saved to product_listing.html")

# To display the HTML directly in the notebook (requires IPython.display):
from IPython.display import HTML
HTML(html_content)
```

Output :



Explanantion : I built a complete contact form web page with fields for Name, Email, and Message. It includes client-side JavaScript validation for empty fields and email format, displaying inline error messages and a success message upon valid submission. We also addressed a SyntaxWarning during its generation.

Task 5: Modal Popup for User Notifications

Scenario

You are building a **user notification popup** for announcements or alerts on a website.

Task Description

- Use AI to help generate:
  - HTML structure for a modal popup
  - CSS styling with a semi-transparent background overlay
  - JavaScript to open and close the modal
- Include multiple ways to close the modal (button or overlay click).

Expected Output

- A web page where:
  - Clicking “Open Modal” displays the popup
  - Popup overlays the page content clearly
  - Modal closes smoothly when instructed

Prompt :

```
You are an expert web developer. Please create a complete web page that includes a modal popup for user notifications. The page should have a button labeled "Open Modal" that, when clicked, displays the popup. The modal should overlay the page content with a semi-transparent background and contain a message along with a close button. Use JavaScript to handle opening and closing the modal, and allow multiple ways to close it (either by clicking the close button or by clicking outside the modal on the overlay). Include clear comments in the JavaScript code to explain the logic. Provide the full HTML, CSS, and JavaScript code together so it can be copied and run directly.
```

Code:

```
html_content = r"""
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Modal Popup Example</title>
    <style>
        /* Basic body styling */
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            display: flex;
            flex-direction: column;
            justify-content: center;
            align-items: center;
            min-height: 100vh;
            margin: 0;
            background-color: #f0f2f5;
            color: #333;
            text-align: center;
        }

        /* Button to open the modal */
        .open-modal-button {
            background-color: #007bff;
            color: white;
            padding: 12px 25px;
            border: none;
```

```
            border-radius: 5px;
            font-size: 1em;
            cursor: pointer;
            transition: background-color 0.3s ease;
        }

        .open-modal-button:hover {
            background-color: #0056b3;
        }

        /* Modal overlay styles */
        .modal-overlay {
            /* Initially hidden */
            display: none;
            /* Position over everything */
            position: fixed;
            top: 0;
            left: 0;
            width: 100%;
            height: 100%;
            /* Semi-transparent black background */
            background-color: rgba(0, 0, 0, 0.7);
            /* Center the modal content */
            display: flex;
            justify-content: center;
            align-items: center;
            /* Ensure it's above other content */
            z-index: 1000;
```

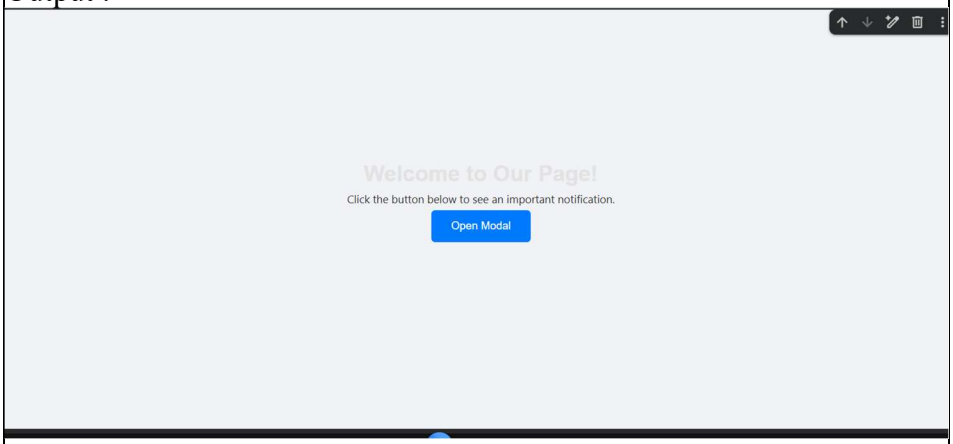
```css
/* Modal overlay visible state */
.modal-overlay.show {
    display: flex;
    opacity: 1;
}

/* Modal content box */
.modal-content {
    background-color: #ffffff;
    padding: 30px;
    border-radius: 8px;
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.3);
    max-width: 500px;
    width: 90%;
    position: relative;
    animation: slideIn 0.3s ease-out;
}

/* Keyframe animation for modal entry */
@keyframes slideIn {
    from { transform: translateY(-50px); opacity: 0; }
    to { transform: translateY(0); opacity: 1; }
}

.modal-content h2 {
    color: #007bff;
    margin-top: 0;
    margin-bottom: 20px;
```

```
            .modal-content p {
                margin-bottom: 25px;
                line-height: 1.6;
            }

            /* Close button for the modal */
            .close-button {
                position: absolute;
                top: 10px;
                right: 15px;
                font-size: 1.5em;
                font-weight: bold;
                color: #aaa;
                cursor: pointer;
                background: none;
                border: none;
                padding: 0;
                line-height: 1;
            }

            .close-button:hover,
            .close-button:focus {
                color: #333;
                outline: none;
            }
        </style>
    </head>
    <body>
```

```
    <h1>Welcome to Our Page!</h1>
    <p>Click the button below to see an important notification.</p>

    <button id="openModalBtn" class="open-modal-button">Open Modal</button>

    <!-- The Modal Structure -->
    <div id="myModal" class="modal-overlay">
        <div class="modal-content">
            <button class="close-button" id="closeModalBtn">&times;</button>
            <h2>Notification!</h2>
            <p>This is an important message for you. Please read it carefully and take necessary action.</p>
            <p>You can close this modal by clicking the 'X' button or by clicking anywhere outside this box.</p>
        </div>
    </div>

    <script>
        document.addEventListener('DOMContentLoaded', function() {
            // Get references to the DOM elements
            const openModalBtn = document.getElementById('openModalBtn');
            const closeModalBtn = document.getElementById('closeModalBtn');
            const myModal = document.getElementById('myModal');
            const modalContent = document.querySelector('.modal-content');

            // Function to open the modal
            function openModal() {
                myModal.classList.add('show'); // Add 'show' class to make the modal visible
                document.body.style.overflow = 'hidden'; // Prevent scrolling the background content
            }
```

```
            // Check if the click occurred directly on the modal-overlay itself,
            // and not on its child elements (like modal-content).
            // This allows closing when clicking on the dimmed background.
            if (event.target === myModal) {
                closeModal();
            }
        });

        // Optional: Close modal with 'Escape' key
        document.addEventListener('keydown', function(event) {
            if (event.key === 'Escape' && myModal.classList.contains('show')) {
                closeModal();
            }
        });
    });
    </script>
</body>
</html>
"""

# To save this as an HTML file, you can uncomment and run the following lines:
# with open('modal_popup.html', 'w') as f:
#     f.write(html_content)
# print("HTML content saved to modal_popup.html")

# To display the HTML directly in the notebook (requires IPython.display):
from IPython.display import HTML
HTML(html_content)
```

Output :



Explanation : I developed a web page with an interactive modal popup. It features an 'Open Modal' button that, when clicked, displays a notification modal. The modal can be closed by clicking its internal 'X' button, by clicking anywhere on the semi-transparent overlay outside the modal content, or by pressing the 'Escape' key.